

Node.js Fundamentals

Professional Classroom Handout | Beginner Level | Topics 1–8

Learning outcomes: Students will understand the Node.js runtime, basic setup and REPL, the browser-vs-Node environment, CommonJS modules, and single/array/object export patterns.

1. Introduction to Node.js

Node.js is a JavaScript runtime that allows JavaScript code to execute outside a web browser. It provides the environment needed to run JavaScript files from a terminal and forms the foundation for server-side JavaScript development.

Examples

Basic example

```
console.log("Hello Node.js");
```

Run the file

```
node app.js
```

Key points

- Node.js is a runtime, not a new programming language.
- The programming language remains JavaScript.
- A .js file can be executed from the terminal using Node.

Worked example

Task: Create app.js and print a student name and course.

```
const name = "Junaid";  
const course = "Node.js";  
console.log(name);  
console.log(course);
```

Practice

1. Create intro.js and print three lines about yourself.
2. Create two variables and print their sum.

2. History & Prerequisites

This section introduces the background of Node.js and the knowledge expected before beginning. At beginner level, the most important prerequisite is a working understanding of JavaScript fundamentals.

Key points

- Recommended foundation: variables, functions, arrays and objects.
- Basic terminal usage is helpful for running Node.js files.

Practice

1. Write one normal JavaScript function and one arrow function.
2. Create an array containing four subjects and an object containing a student's name and course.

3. Installation and Running Node.js

After Node.js is installed, JavaScript files can be executed from a terminal. A code editor such as VS Code can be used to create and manage project files.

Examples

Typical workflow

```
Create app.js
Open terminal in the project folder
Run: node app.js
```

Key points

- The terminal should point to the correct project folder.
- Read terminal errors carefully; they often identify the file and line causing the problem.

Practice

1. Create test.js, print 'Node is working', and execute it from the terminal.

4. Node.js REPL

REPL is an interactive environment for entering JavaScript expressions and receiving immediate results. It is useful for quick experiments and checking small expressions.

Examples

REPL example

```
node
> 10 + 5
15
> 8 * 4
32
```

Key points

- Use REPL for quick tests.
- Use saved .js files for larger or reusable programs.

Worked example

Task: Evaluate 25 - 10 in REPL.

```
node
> 25 - 10
15
```

Practice

1. Use REPL to test addition, multiplication and a string variable.

5. Browser Environment vs Node.js Environment

Browser JavaScript and Node.js execute in different environments. Browsers provide web-page-related objects such as window and document. Node.js is not a browser and provides its own runtime environment, including a global object.

Examples

Concept map

```
Browser environment -> window, document
Node.js environment -> global runtime environment
```

Key points

- Do not expect browser-only page objects to behave as normal Node.js globals.
- The main idea is to understand that the execution environments are different.

Practice

1. Explain in one sentence why document belongs to browser-based JavaScript rather than a basic Node.js script.

6. CommonJS Modules

A module separates a program into reusable files. In the CommonJS style practiced here, `module.exports` exposes a value from one file and `require()` loads that exported value in another file.

Examples

Export file: math.js

```
const add = (a, b) => {
  return a + b;
};

module.exports = add;
```

Import file: app.js

```
const add = require("./math");
console.log(add(10, 5)); // 15
```

Key points

- `module.exports` = export from a module.
- `require()` = load an exported module.
- `./` refers to the current folder in a relative path.
- The `.js` extension is commonly omitted in `require('./math')`.

Worked example

Task: Create a subtraction module and use it in `app.js`.

```
// sub.js
const sub = (a, b) => a - b;
module.exports = sub;

// app.js
const sub = require("./sub");
console.log(sub(10, 2)); // 8
```

Practice

1. Create `square.js`, export `square(number)`, and use it in `app.js`.
2. Create `greeting.js`, export `welcome(name)`, and use it in `app.js`.

7. Single, Array and Object Exports

Exports can be organized in different forms. The examples below use the CommonJS module system.

Examples

Single export

```
module.exports = add;
const add = require('./math');
```

Array export

```
module.exports = [add, sub];
const [add, sub] = require('./math');
```

Object export

```
module.exports = { add, sub };
const { add, sub } = require('./math');
```

Key points

- `[]` represents an array.
- `{}` represents an object.
- Array destructuring uses `[]`.
- Object destructuring uses `{}`.
- An imported object can also be used as `math.add()`.

Worked example

Task: Export add and sub in an object and use both.

```
// math.js
module.exports = { add, sub };

// app.js
const math = require("./math");
console.log(math.add(20, 10));
console.log(math.sub(20, 10));
```

Practice

1. Export multiply and divide in an array and destructure them.
2. Export add, sub, multiply and divide in an object and destructure them.

8. Named/Aggregate Export Pattern in Current Practice

Related functions can be grouped in one exported object. This gives the importing program named properties such as add, sub, multi and div. Object destructuring can then select those functions directly.

Examples

calculator.js

```
const add = (a,b) => a+b;
const sub = (a,b) => a-b;
const multi = (a,b) => a*b;
const div = (a,b) => a/b;

module.exports = { add, sub, multi, div };
```

Import complete object

```
const math = require("./calculator");
console.log(math.add(20, 10));
```

Destructure object

```
const { add, sub, multi, div } = require("./calculator");
console.log(add(20, 10));
```

Key points

- Grouping related functions keeps a module organized.

- Destructuring provides direct access to selected object properties.

Practice

1. Add remainder(a,b) to calculator.js and export it with the other functions.
2. Import only add and remainder using object destructuring.

Classroom Mini Project: Student Utility System

Create four files: heading.js (single export), subject.js (array export), math.js (object export), and app.js (main importing file). The goal is to combine all export patterns in one small program.

Extension: Add percentage.js with a single exported percentage(obtained, total) function and use it in app.js.

Assessment Questions

1. What is Node.js?
2. What does module.exports do?
3. What does require() do?
4. What does ./ mean in require('./math')?
5. What is the difference between [] and {}?
6. How do array and object destructuring differ?
7. How can add be called when the entire math object is imported?